



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS
PROFESIONALES
ZONA MEDIA

INGENIERÍA MECATRÓNICA



PRÁCTICA 5

Uso de los pines de propósito general de entrada/salida en
Raspberry Pi Pico W Usando C

MATERIA: MICROCONTROLADORES

DOCENTE: Ing. JESÚS PADRÓN

ALUMNO: CARLOS ANGEL GARCÍA SIFUENTES

CLAVE: A383917

CARRERA: INGENIERÍA MECATRÓNICA

SEMESTRE: 4° SEMESTRE

FECHA: 19 DE MAYO DE 2026

Parpadeo del LED Integrado a la Raspberry Pi Pico W

Mediante Programación en Lenguaje C

CARLOS ÁNGEL GARCÍA SIFUENTES

a383917@alumnos.uaslp.mx

Fecha: 19 de mayo de 2026

Resumen—Se presenta el desarrollo de la Práctica 5, cuyo objetivo fue aprender el uso de los pines de propósito general de entrada/salida (GPIO) de la Raspberry Pi Pico W mediante la implementación de un programa en lenguaje C capaz de controlar el parpadeo de un LED externo. Se configuró un pin GPIO como salida digital utilizando la librería del SDK oficial de Raspberry Pi, se compiló el proyecto mediante CMake y Visual Studio Code, y se verificó el funcionamiento del circuito mediante la programación del microcontrolador con un archivo .uf2. Además, se analizaron los criterios de cálculo de resistencias limitadoras y el concepto de PWM aplicado al control de LEDs.

I. INTRODUCCIÓN

I-A Objetivo del ejercicio

El objetivo principal de esta práctica es aprender a crear un proyecto en C capaz de programar la Raspberry Pi Pico W usando como referencia el código de ejemplo que permite el parpadeo del LED desde una GPIO en la Raspberry Pi Pico W. Para lograrlo, se utiliza el SDK oficial de Raspberry Pi junto con las librerías `pico_stdlib` y `pico_cyw43_arch_none`.

Durante la práctica se exploró la estructura básica de un proyecto para Raspberry Pi Pico W y la interacción entre el SDK y el hardware integrado.

I-B Importancia del SDK de Raspberry Pi

El SDK (*Software Development Kit*) de la Raspberry Pi proporciona un conjunto de herramientas, librerías y ejemplos que simplifican el desarrollo de aplicaciones embebidas para la familia de microcontroladores RP2040. Entre sus ventajas principales destacan:

- **Abstracción del hardware:** permite controlar periféricos como GPIO, UART, SPI e I2C sin escribir código de bajo nivel para cada registro.
- **Portabilidad:** los proyectos compilados con el SDK son compatibles con la mayoría de placas basadas

en RP2040 que utilicen soporte del SDK.

- **Integración con CMake:** facilita la configuración y compilación de proyectos de manera estructurada y reproducible.
- **Soporte para el chip CYW43439:** en la versión Pico W, el SDK incluye librerías dedicadas para controlar el módulo Wi-Fi/Bluetooth y los GPIO asociados a él, como el LED integrado.

I-C Aplicaciones del Raspberry Pi Pico W en microcontroladores

La Raspberry Pi Pico W, basada en el microcontrolador RP2040, se posiciona como una plataforma versátil para aplicaciones de IoT (*Internet of Things*), automatización y sistemas embebidos. Algunas de sus aplicaciones más relevantes en el ámbito de la ingeniería mecatrónica incluyen el control de actuadores y sensores en tiempo real, la comunicación inalámbrica mediante Wi-Fi o Bluetooth, el procesamiento de señales analógicas gracias a su ADC de 12 bits, y la implementación de protocolos de comunicación industrial como UART, SPI e I2C. La programación en C con el SDK permite aprovechar al máximo el rendimiento del núcleo Cortex-M0+ a 133 MHz, siendo ideal para tareas de control de precisión.

II. DESARROLLO

II-A Configuración del Entorno de Desarrollo

II-A1. Instalación y configuración del SDK

El primer paso para el desarrollo del proyecto consiste en la instalación del SDK de Raspberry Pi en el equipo de trabajo. Este SDK se descarga desde el repositorio oficial y se instala junto con las herramientas de compilación cruzada (*arm-none-eabi-gcc*) y CMake. Una vez instalado, el sistema crea automáticamente las variables de entorno necesarias, tales como `PICO_SDK_PATH` y `PICO_EXAMPLES_PATH`, que son utilizadas posteriormente en los comandos de configuración del proyecto.

Para comenzar el trabajo, se abre la aplicación **Pico Developer PowerShell** con privilegios de administrador. Esta terminal tiene preconfiguradas todas las rutas y variables de entorno requeridas por el SDK, lo que simplifica significativamente el proceso de desarrollo.

II-A2. Descripción de las librerías utilizadas

pico_stdlib: Es la librería estándar del SDK de Raspberry Pi. Incluye funciones básicas de entrada/salida, temporización (`sleep_ms`, `sleep_us`) y control de GPIO, siendo suficiente para aplicaciones sencillas como el control de LEDs mediante salidas digitales.

II-A3. Creación de la estructura de carpetas y archivos

Se crea una carpeta principal llamada **MICROCONTROLADORES** para alojar todas las prácticas del curso. Dentro de ella, se genera la carpeta **PRACTICA_5** (sin espacios en blanco). A continuación, en la terminal Pico Developer PowerShell, se navega a dicha carpeta con el comando `cd` y se copian los archivos base del SDK ejecutando:

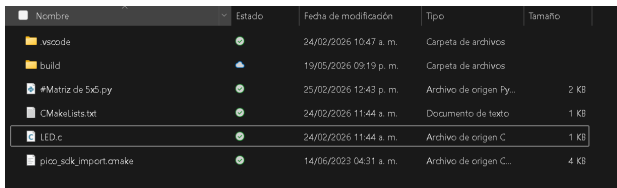
COMANDOS POWERSHELL — CONFIGURACIÓN INICIAL

```
cd C:\Users\Carlos\MICROCONTROLADORES\
PRACTICA_5

copy ${env:PICO_SDK_PATH}\external\
pico_sdk_import.cmake .

copy ${env:PICO_EXAMPLES_PATH}\.vscode . -
recurse
```

Al ejecutar estos comandos correctamente, la carpeta del proyecto contendrá el archivo `pico_sdk_import.cmake` y la subcarpeta `.vscode` con la configuración para Visual Studio Code.



Nombre	Estado	Fecha de modificación	Tipo	Tamaño
.vscode		24/02/2025 10:47 a. m.	Carpeta de archivos	
build		19/05/2025 09:19 p. m.	Carpeta de archivos	
#Matriz de 5x5.py		25/02/2025 12:43 p. m.	Archivo de origen Py...	2 KB
CMakeLists.txt		24/02/2025 11:44 a. m.	Documento de texto	1 KB
LED.c		24/02/2025 11:44 a. m.	Archivo de origen C...	1 KB
pico_sdk_import.cmake		14/06/2023 04:31 a. m.	Archivo de origen C...	4 KB

Figura 1: Estructura inicial de la carpeta **Práctica_4** tras copiar los archivos base del SDK.

II-B Creación del Proyecto

II-B1. Archivo CMakeLists.txt

El archivo `CMakeLists.txt` es el corazón del sistema de compilación CMake. Define el nombre del proyecto, las fuentes a compilar y las librerías que se deben

enlazar. Para esta práctica, su contenido es el siguiente:

ARCHIVO CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)
set(PICO_BOARD pico_w)

Project(PRACTICA_5)

pico_sdk_init()

add_executable(LED
    LED.c
)

target_link_libraries(LED
    pico_stdlib
)

pico_add_extra_outputs(LED)
```

Los campos más relevantes son:

- `add_executable`: especifica el nombre del ejecutable (`LED`) y el archivo fuente (`LED.c`).
- `target_link_libraries`: enlaza las librerías necesarias con el ejecutable.
- `pico_add_extra_outputs`: genera los archivos de salida adicionales, incluido el `.uf2` necesario para programar la placa.

II-B2. Implementación del código en C

Una vez configurado el sistema de compilación, se crea el archivo `LED.c` con el siguiente código, que implementa el control del LED conectado a GPIO mediante un bucle infinito:

CÓDIGO C — PARPADEO DEL LED EN GPIO-OUT

```
#include "pico/stdlib.h"

int main(){
    const uint LED_PIN = 3;

    gpio_init(LED_PIN);
    gpio_set_dir(LED_PIN, GPIO_OUT);

    while(true){
        gpio_put(LED_PIN, 1);
        sleep_ms(250);
        gpio_put(LED_PIN, 0);
        sleep_ms(250);
    }
}
```

La lógica del programa es la siguiente:

- `gpio_init(LED_PIN)` inicializa el GPIO seleccionado.
- `gpio_set_dir(LED_PIN, GPIO_OUT)` configura el pin como salida digital.
- `gpio_put()` cambia el estado lógico del GPIO para encender y apagar el LED.
- El ciclo `while(true)` ejecuta indefinidamente el parpadeo con retardos de 250 ms.

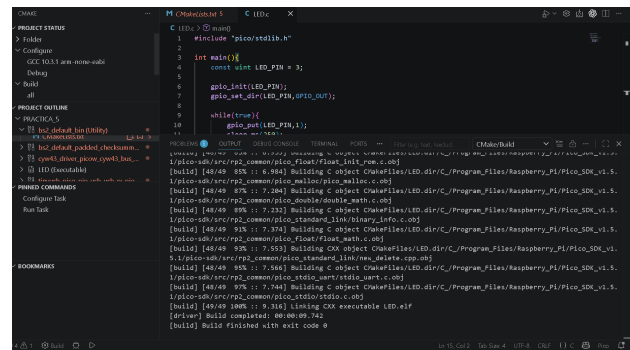


Figura 3: Terminal de Visual Studio Code mostrando la compilación exitosa del proyecto.

```
1 #include "pico/stdlib.h"
2
3 int main(){
4     const uint LED_PIN = 3;
5
6     gpio_init(LED_PIN);
7     gpio_set_dir(LED_PIN, GPIO_OUT);
8
9     while(true){
10         gpio_put(LED_PIN, 1);
11         sleep_ms(250);
12         gpio_put(LED_PIN, 0);
13         sleep_ms(250);
14     }
15 }
```

Figura 2: Código `LED.c` visualizado en Visual Studio Code con la extensión Pico.

II-C Proceso de Compilación y Carga del Programa

II-C1. Compilación en Visual Studio Code

Con el proyecto completamente configurado, se abre la aplicación **Pico Visual Studio Code**. Dentro del editor se realiza la siguiente secuencia:

1. **Abrir la carpeta del proyecto:** mediante *File* → *Open Folder*, se selecciona la carpeta `PRACTICA_5`.
2. **Confiar en el autor:** VS Code solicita confirmación de confianza en los archivos; se selecciona *Yes, I trust the authors*.
3. **Seleccionar el kit de compilación:** se elige *Pico ARM GCC* en el menú de selección de kits de CMake.
4. **Compilar el ejecutable:** en el panel de CMake (*Project Outline*), se posiciona el cursor sobre *Parpadeo (Executable)* y se presiona el ícono de compilación (*Build*).

El proceso de compilación puede observarse en la terminal integrada de VS Code. Una compilación exitosa genera el mensaje `Build finished with exit code 0` y crea la carpeta `build` dentro del directorio del proyecto.

II-C2. Generación del archivo `.uf2` y transferencia al Pico W

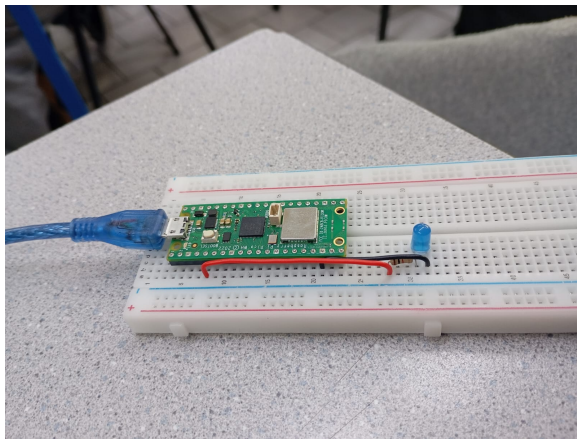
Dentro de la carpeta `build` se generan múltiples archivos de salida. El archivo de interés es `LED.uf2`, que es el formato estándar para programar placas Raspberry Pi Pico mediante arrastrar y soltar (*drag and drop*).

El procedimiento de carga al microcontrolador es:

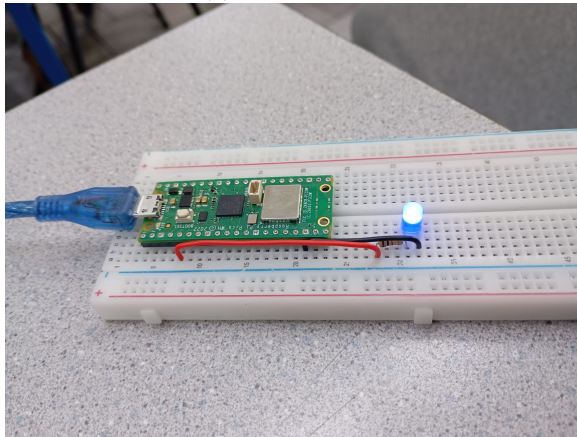
1. Mantener presionado el botón **BOOTSEL** de la Raspberry Pi Pico W.
2. Conectar el cable micro USB al ordenador sin soltar el botón.
3. Soltar el botón **BOOTSEL**. La placa aparecerá como un dispositivo USB de almacenamiento masivo llamado `RPI-RP2`.
4. Arrastrar el archivo `Parpadeo.uf2` a la unidad `RPI-RP2`.
5. La placa se reinicia automáticamente y comienza a ejecutar el programa.

II-C3. Verificación del funcionamiento

Tras la transferencia del archivo `.uf2`, el LED integrado de la Raspberry Pi Pico W comienza a parpadear de manera continua con un período de 500 ms (encendido 250 ms, apagado 250 ms), confirmando el correcto funcionamiento del programa.



(a) LED apagado



(b) LED encendido

Figura 4: Estados del LED durante el parpadeo.

III. RESULTADOS

III-A Evidencia del código utilizado

El código implementado en `LED.c` fue compilado sin errores ni advertencias usando el toolchain `arm-none-eabi-gcc` incluido en el SDK oficial de Raspberry Pi. La compilación generó los archivos de la Tabla I.

Tabla I: Archivos generados en la carpeta `build` tras la compilación

Archivo	Descripción
<code>LED.elf</code>	Ejecutable ELF para depuración
<code>LED.uf2</code>	Binario para programar la placa
<code>LED.bin</code>	Imagen binaria plana
<code>LED.hex</code>	Formato Intel HEX
<code>LED.elf.map</code>	Mapa de memoria del programa
<code>LED.dis</code>	Desensamblado del código

III-B Capturas del proceso

Las figuras 1, 2, 3, y 4 documentan cada etapa del proceso: configuración del entorno, escritura del código, compilación, modo de programación del dispositivo y verificación del funcionamiento, respectivamente.

III-C Errores encontrados y soluciones

Durante el desarrollo se identificaron los siguientes posibles puntos de falla y sus soluciones:

- **Fallo al conectar como estaba en el diagrama.** Uno de los errores más comunes que cometí fue el error de conexiones de los pines correctos en la protoboard
- **El LED no parpadea al cargar el .uf2.** Causa: el botón BOOTSEL no se mantuvo presionado durante la conexión USB. Solución: repetir el procedimiento de conexión correctamente.
- **VS Code no detecta el kit Pico ARM GCC.** Causa: la aplicación utilizada fue VS Code genérico en lugar de *Pico Visual Studio Code*. Solución: abrir la versión correcta del editor o instalar la extensión CMake Tools.

IV. PREGUNTAS DE REPASO

IV-A ¿Cuáles son los pasos necesarios para configurar un pin GPIO como salida en el Raspberry Pi Pico W si queremos controlar un LED?

Para controlar un LED mediante un pin GPIO del Raspberry Pi Pico W es necesario realizar una serie de pasos utilizando la librería `pico_stdlib`. Primero, se debe seleccionar el pin GPIO que será utilizado mediante una constante o variable. Posteriormente, se inicializa dicho pin con la función `gpio_init()`, la cual habilita el acceso al GPIO seleccionado.

Después de la inicialización, el pin debe configurarse como salida digital usando la función `gpio_set_dir()` junto con el parámetro `GPIO_OUT`. Finalmente, el estado lógico del pin puede ser controlado mediante la función `gpio_put()`, estableciendo un valor alto (1) para encender el LED o bajo (0) para apagarlo.

En resumen, la secuencia básica es:

1. Seleccionar el pin GPIO.
2. Inicializarlo con `gpio_init()`.
3. Configurarlo como salida usando `gpio_set_dir()`.
4. Controlar su estado mediante `gpio_put()`.

IV-B ¿Cuál es el valor ideal de resistencia que se necesita para hacer que un LED rojo cuya caída de voltaje es 1.5V si queremos tener una corriente fija de 15mA? Muestra tus cálculos mediante una fotografía de tu procedimiento.

Para calcular la resistencia necesaria se utiliza la Ley de Ohm, considerando el voltaje de alimentación del GPIO del Raspberry Pi Pico W (3.3 V), la caída de voltaje del LED rojo (1.5 V) y la corriente deseada (15 mA).

La expresión utilizada es:

$$R = \frac{V_{fuente} - V_{LED}}{I}$$

Sustituyendo los valores:

$$R = \frac{3.3 - 1.5}{0.015}$$

$$R = 120 \Omega$$

Por lo tanto, el valor ideal de resistencia requerido para mantener una corriente aproximada de 15 mA es de 120 Ω .

IV-C ¿Cuál es el valor comercial de resistencia más óptimo para acercarse lo más posible a la corriente requerida en el paso anterior?

Los valores de resistencia disponibles comercialmente se encuentran organizados en series normalizadas, como E6, E12 y E24. En este caso, el valor calculado de 120 Ω coincide directamente con un valor comercial ampliamente disponible.

Por esta razón, la resistencia comercial más adecuada es una de 120 Ω , ya que permite obtener aproximadamente la corriente requerida de 15 mA sin necesidad de aproximaciones adicionales. Sin embargo, en la práctica se empleó una resistencia de 1 k Ω , la cual limita aún más la corriente y reduce la intensidad luminosa del LED, manteniendo un funcionamiento seguro del circuito.

IV-D ¿Qué es el PWM y qué función tiene cuando trabajamos con LEDs?

PWM (*Pulse Width Modulation* o Modulación por Ancho de Pulso) es una técnica utilizada para controlar la potencia promedio entregada a un dispositivo electrónico mediante la variación del tiempo que una señal permanece en estado alto respecto a su período total, conocido como ciclo de trabajo (*duty cycle*).

Cuando se trabaja con LEDs, el PWM permite controlar el brillo sin modificar directamente el voltaje de alimentación. Un ciclo de trabajo pequeño produce un brillo tenue, mientras que un ciclo de trabajo grande genera mayor intensidad luminosa. Esta técnica es ampliamente utilizada porque ofrece un control eficiente y preciso del nivel de iluminación.

V. CONCLUSIONES

- La Raspberry Pi Pico W, programada en C mediante el SDK oficial, permite desarrollar aplicaciones embebidas de manera estructurada y eficiente. El proceso de configuración del entorno, aunque requiere varios pasos iniciales, es reproducible y bien documentado.
- El uso del archivo `CMakeLists.txt` resulta fundamental para la gestión del proyecto. Una configuración incorrecta de este archivo, como omitir `set(PICO_BOARD pico_w)`, impide el acceso a las librerías del chip CYW43439 y por tanto al LED integrado.
- El uso de la librería `pico_stdlib` permitió controlar un pin GPIO de manera sencilla y estructurada, evidenciando la utilidad del SDK oficial para aplicaciones básicas de entrada y salida digital.
- El formato UF2 simplifica enormemente el proceso de programación del microcontrolador al eliminar la necesidad de herramientas externas, haciendo al Raspberry Pi Pico W una plataforma muy accesible para el aprendizaje y prototipado rápido.
- Como mejora futura, se podría parametrizar el intervalo de parpadeo mediante una variable en lugar de usar valores fijos de 250 ms, facilitando ajustes sin modificar múltiples líneas del código. Igualmente, se podría agregar comunicación UART para monitorear el estado del LED en tiempo real desde una terminal serial.